



Aligning with Human Preferences

Hanoi University of Science and Technology

ONE LOVE. ONE FUTURE.

- **Large language models (e.g., GPT):**
 - Pre-trained on a vast textual corpus to predict subsequent tokens.
 - Equip LLMs with world knowledge
 - Facilitate the generation of coherent and fluent text in response to various input
- **Limitations**
 - Not always adept at interpreting a wide range of instructions
 - Can produce biased/toxic content or invent facts
- **Recent research:**
 - Empowering LLM to understand instructions and **align with human expectations**

1. RLHF: Reinforcement Learning From Human

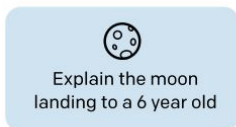


RLHF: Overview

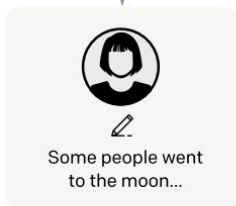
Step 1

**Collect demonstration data,
and train a supervised policy.**

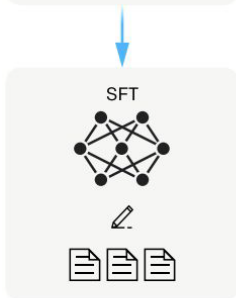
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



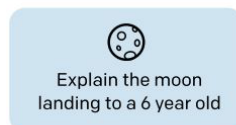
This data is used
to fine-tune GPT-3
with supervised
learning.



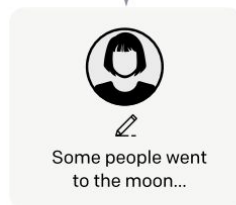
Step 1

**Collect demonstration data,
and train a supervised policy.**

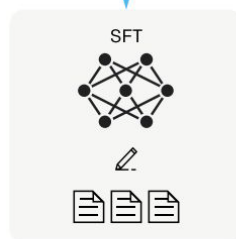
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



This data is used
to fine-tune GPT-3
with supervised
learning.



Step 2

**Collect comparison data,
and train a reward model.**

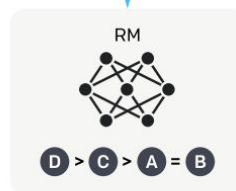
A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.



This data is used
to train our
reward model.

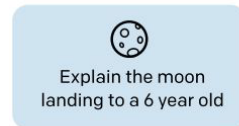


RLHF: Overview

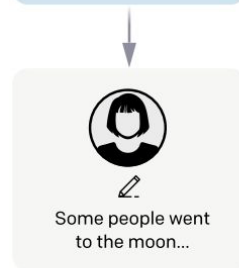
Step 1

Collect demonstration data, and train a supervised policy.

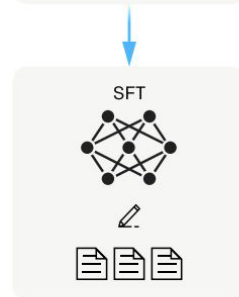
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



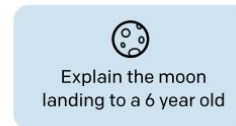
This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

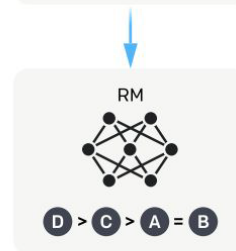
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



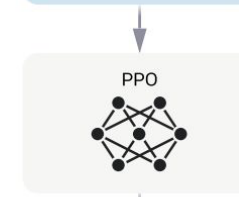
Step 3

Optimize a policy against the reward model using reinforcement learning.

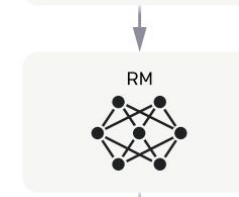
A new prompt is sampled from the dataset.



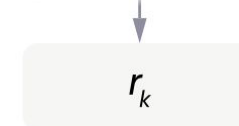
The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.

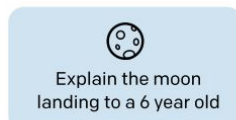


RLHF: Train a Supervised Policy from Demonstration Data

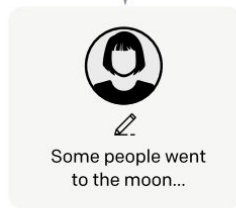
Step 1

Collect demonstration data, and train a supervised policy.

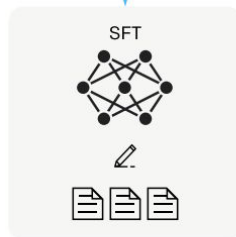
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



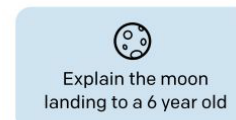
This data is used to fine-tune GPT-3 with supervised learning.



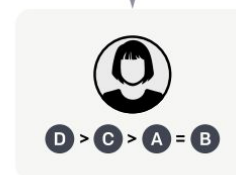
Step 2

Collect comparison data, and train a reward model.

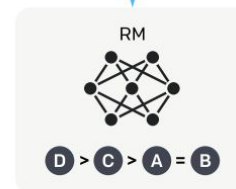
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



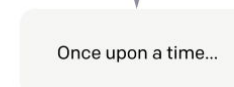
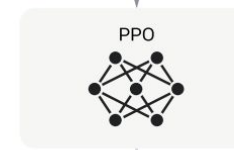
Step 3

Optimize a policy against the reward model using reinforcement learning.

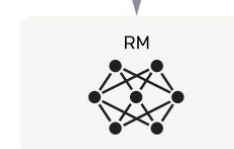
A new prompt is sampled from the dataset.



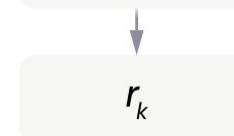
The policy generates an output.



The reward model calculates a reward for the output.



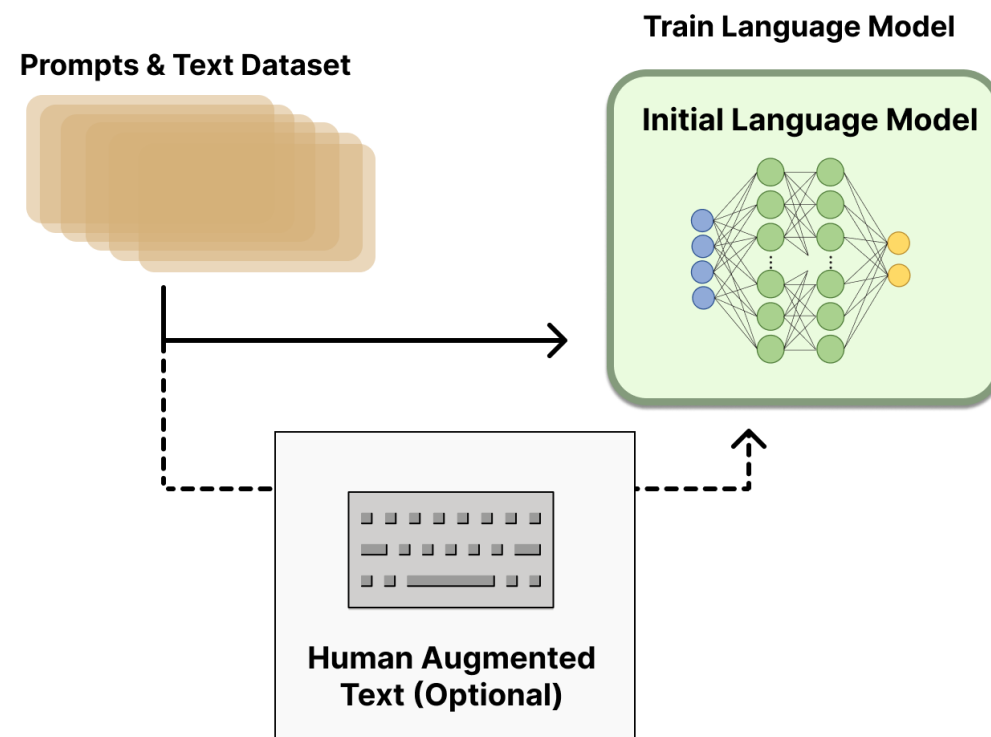
The reward is used to update the policy using PPO.



RLHF: Train a Supervised Policy from Demonstration Data

- Firstly, hiring a team of 40 contractors to label data, based on their performance on a screening test.
- Then collecting a dataset of **human-written demonstrations of the desired output behavior** on (mostly English) prompts submitted to the OpenAI API and some labeler-written prompts,
- Use this dataset to train their supervised learning baselines.

Supervised Fine-tuning

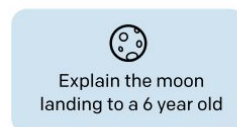


RLHF: Learning a **Reward Model** from Human Feedback

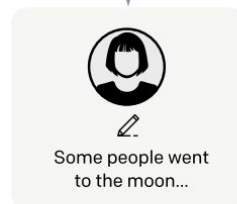
Step 1

Collect demonstration data, and train a supervised policy.

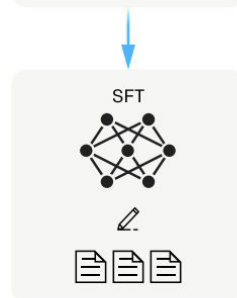
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



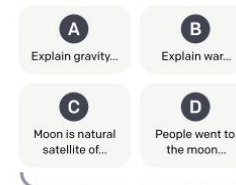
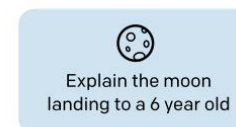
This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

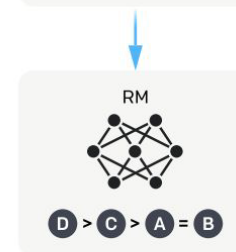
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



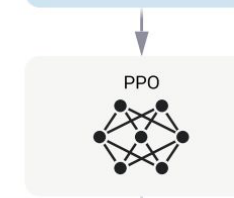
Step 3

Optimize a policy against the reward model using reinforcement learning.

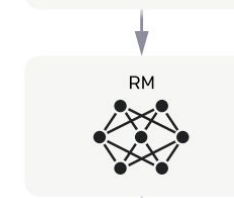
A new prompt is sampled from the dataset.



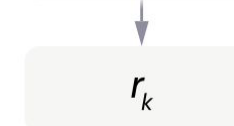
The policy generates an output.



The reward model calculates a reward for the output.

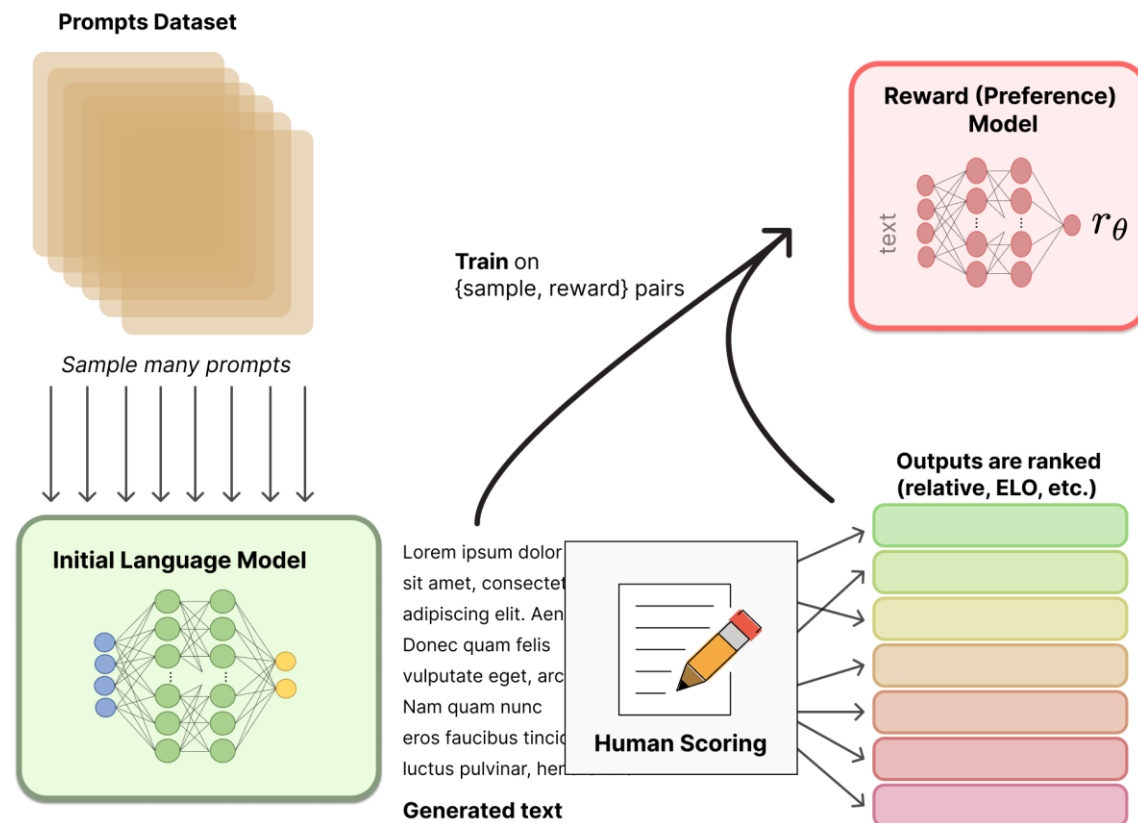


The reward is used to update the policy using PPO.



RLHF: Learning a **Reward Model** from Human Feedback

- Collect a dataset of **human-labeled comparisons** between outputs from OpenAI's models on a larger set of API prompts.
- And then train a reward model (RM) on this dataset to **predict which model output their labelers would prefer**.



RLHF: Learning a **Reward Model** from Human Feedback

- Feedback comes as preferences over model samples:

$$\mathcal{D} = \{x^i, y_w^i, y_l^i\}$$

Prompt Preferred response Dis-preferred response

- Bradley-Terry model connects rewards to preferences

Sigmoid function

$$p(y_w \succ y_l) = \sigma(r(x, y_w) - r(x, y_l)) = \frac{\exp[r(x, y_w)]}{\exp[r(x, y_w)] + \exp[r(x, y_l)]}$$

Rewards assigned to preferred and dis-preferred responses

RLHF: Learning a **Reward Model** from Human Feedback

- Bradley-Terry Model connects rewards to preferences

Sigmoid function

$$p(y_w \succ y_l) = \sigma(r(x, y_w) - r(x, y_l)) = \frac{\exp[r(x, y_w)]}{\exp[r(x, y_w)] + \exp[r(x, y_l)]}$$

Rewards assigned to **preferred** and **dis-preferred** responses

- Train the reward model by minimizing negative log likelihood

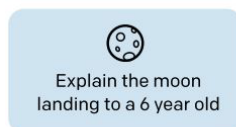
$$\mathcal{L}_R(\phi, \mathcal{D}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(r_\phi(x, y_w) - r_\phi(x, y_l) \right) \right]$$

RLHF: Learning a Policy that Optimize the Reward

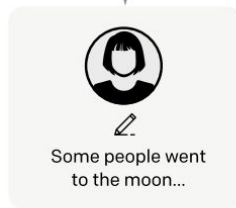
Step 1

Collect demonstration data, and train a supervised policy.

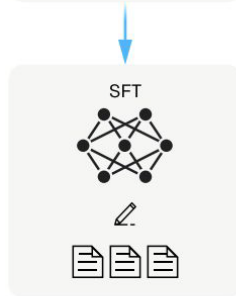
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



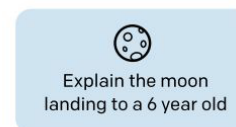
This data is used to fine-tune GPT-3 with supervised learning.



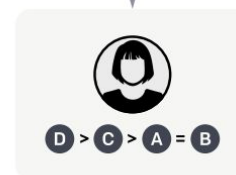
Step 2

Collect comparison data, and train a reward model.

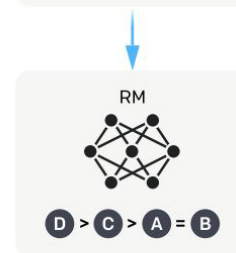
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



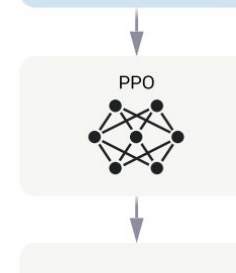
Step 3

Optimize a policy against the reward model using reinforcement learning.

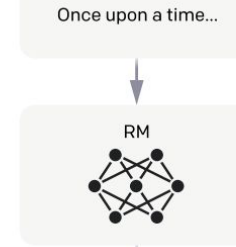
A new prompt is sampled from the dataset.



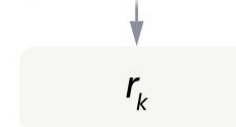
The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.

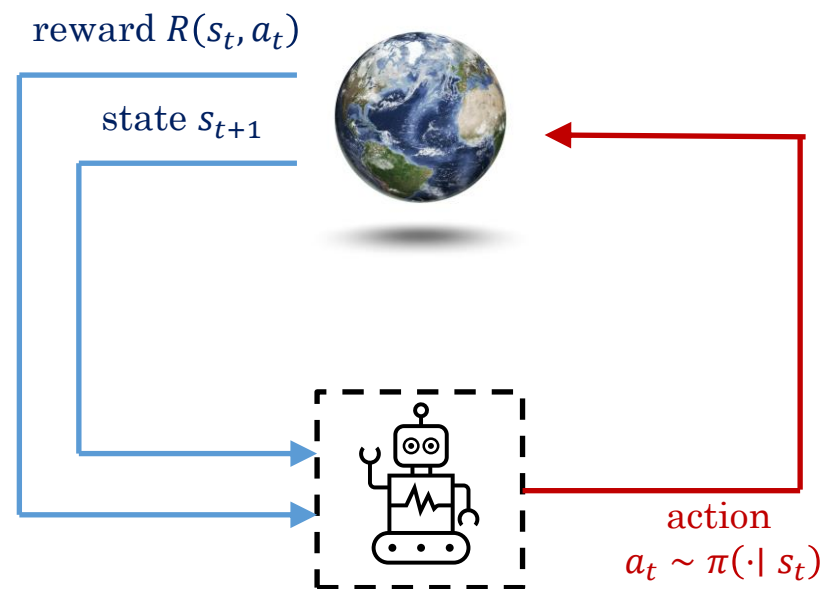


Background on Reinforcement Learning

- **Markov-Decision Process** setup
 - States: S
 - Actions: A
 - Transitions: $P(s' | s, a)$ (unknown)
 - Reward function: $R(s, a)$ (unknown)
- **Goal**: find an optimal policy $\pi_{\theta}(\cdot | s), \forall s$

$$\max_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]$$

- Where $\tau = (s_0, a_0, s_1, a_1, \dots)$



Background on Reinforcement Learning

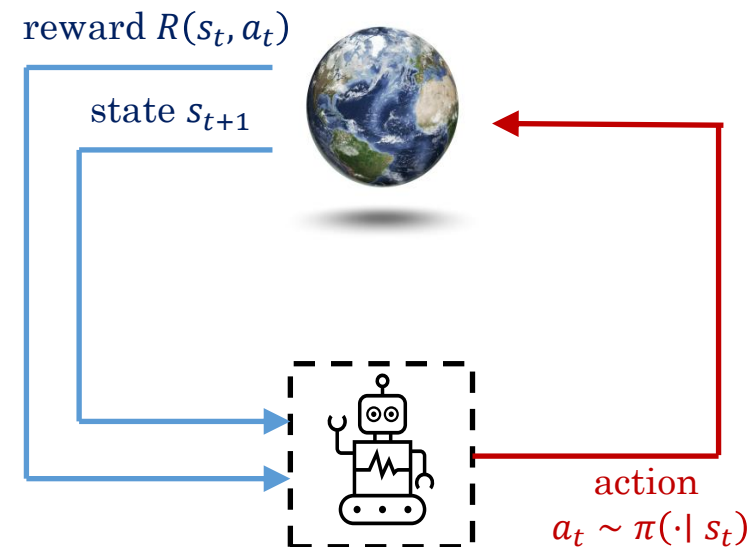
- **Goal:** find an optimal policy $\pi_{\theta}(\cdot | s), \forall s$

$$\max_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]$$

Where $\tau = (s_0, a_0, s_1, a_1, \dots)$

- Some important notions:

- State-action value function $Q_{\pi}(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) V_{\pi}(s')$
- Value function $V_{\pi}(s) = \sum_a \pi(a | s) Q_{\pi}(s, a)$
- Advantage function: $A_{\pi}(s, a) = Q_{\pi}(s, a) - V_{\pi}(s)$



Background on Reinforcement Learning

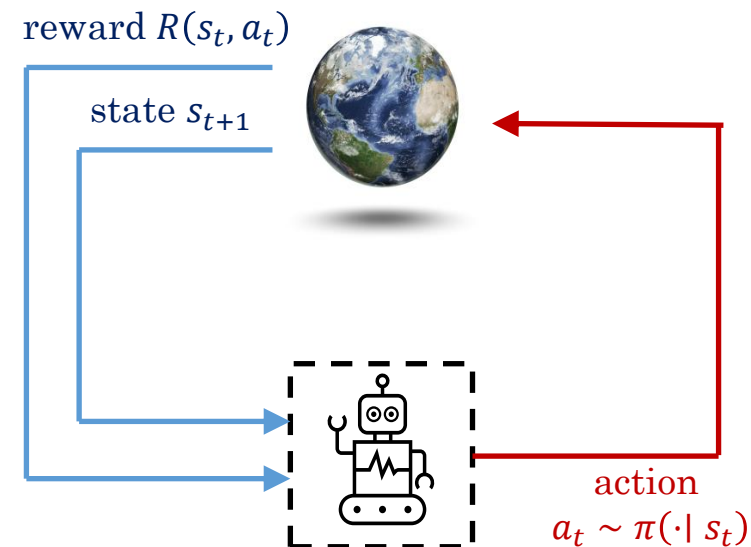
- Goal: find an optimal policy $\pi_{\theta}(\cdot | s), \forall s$

$$\max_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]$$

- Where $\tau = (s_0, a_0, s_1, a_1, \dots)$
- Policy gradient theorem

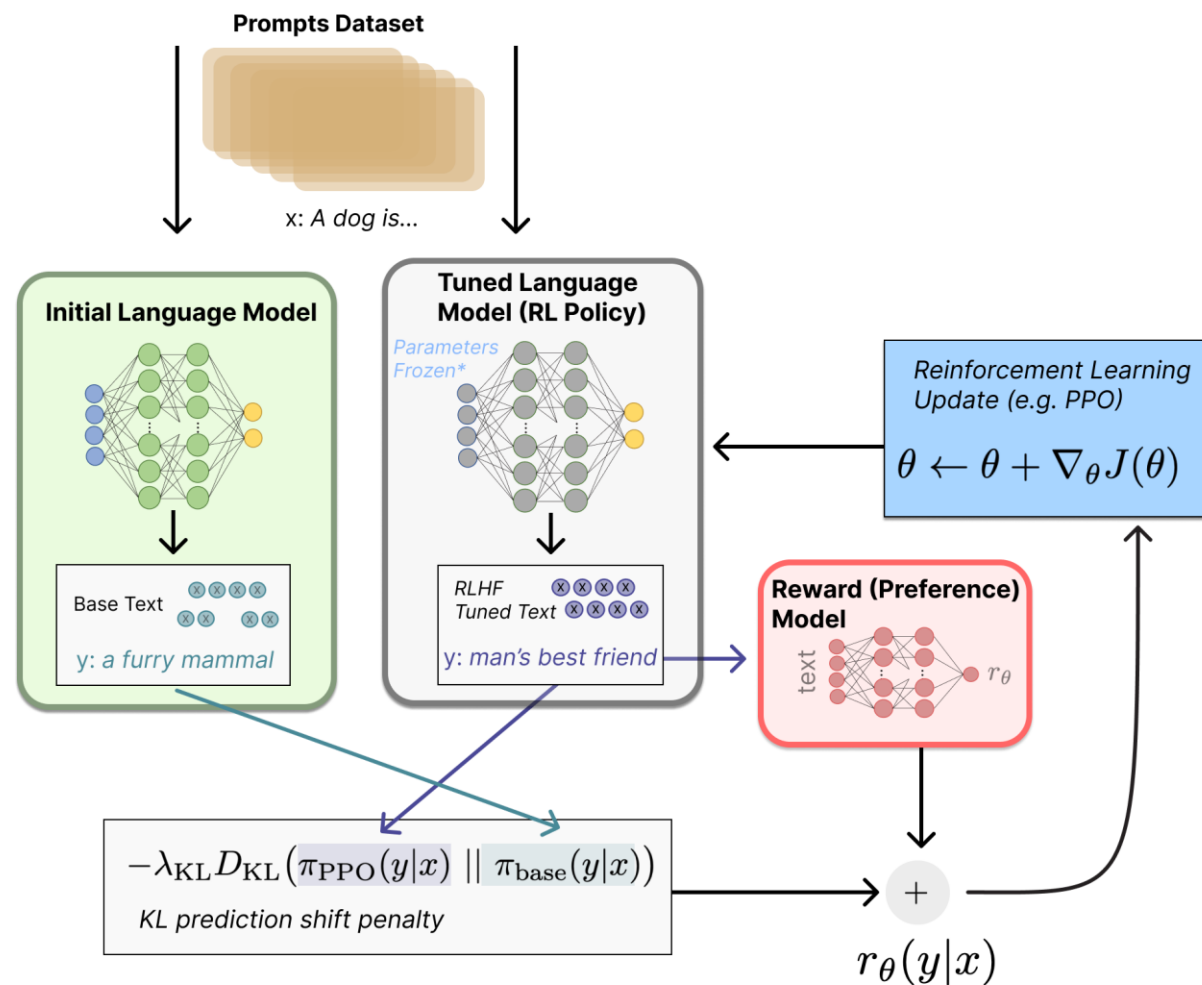
$$\nabla J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\nabla \ln \pi_{\theta}(a | s) Q_{\pi_{\theta}}(s, a) \right]$$

- Policy optimization: gradient ascent
 - **Challenge:** unstable training



RLHF: Learning a **Policy** that Optimize the **Reward**

- Use this **RM** as a reward function and **fine-tune supervised learning baseline** to maximize this reward using the **PPO algorithm**.



RLHF: Learning a **Policy** that Optimize the **Reward**

- Now we have a reward model r_ϕ that represents goodness according to humans
- Next, learn a policy π_θ achieving a high reward
- **Objective**

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}} [r_{\phi}(x, y)]$$

Sample from policy

Want high rewards

RLHF: Learning a **Policy** that Optimize the **Reward**

- Now we have a reward model r_ϕ that represents goodness according to humans
- Next, learn a policy π_θ achieving a high reward **while staying close to original model π_{ref}**
- Objective

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta} [r_\phi(x, y)] - \beta \mathbb{D}_{KL} [\pi_\theta(y | x) \parallel \pi_{ref}(y | x)]$$

Sample from policy

Want high rewards

But keep KL to original model small

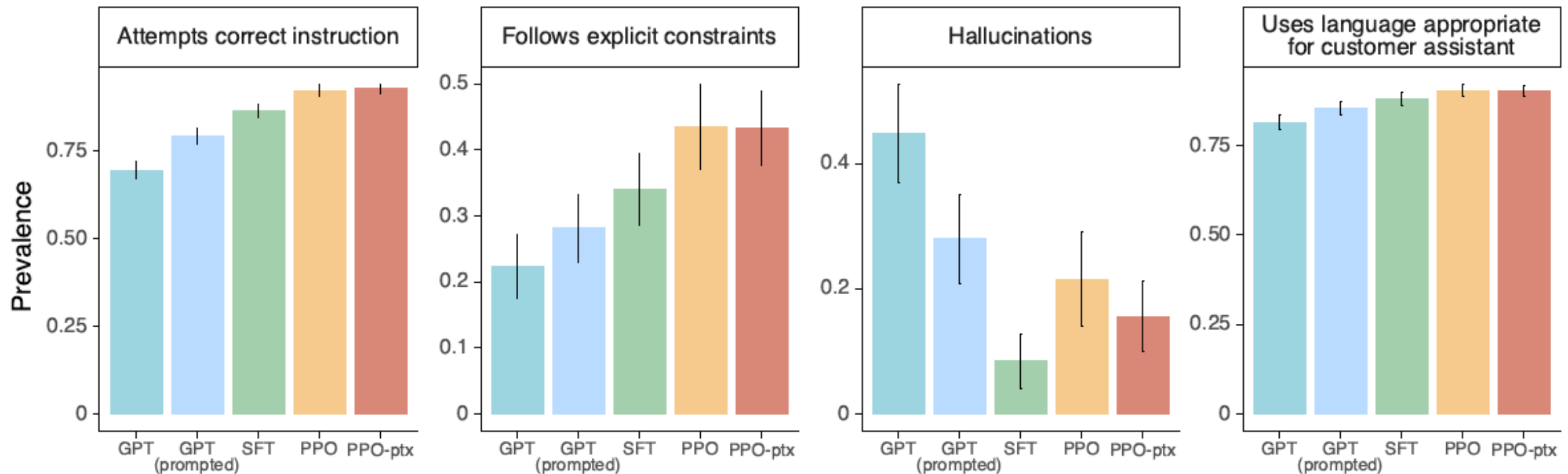
- API distribution
 - Main metric is human preference ratings on a held out set of prompts from the same source as their training distribution.
- Public NLP datasets
 - They evaluate on two types of public datasets, which are FLAN and TO, both consist of a variety of NLP tasks. Also, conduct human evaluations of toxicity on the RealToxicityPrompts dataset

Table 3: Labeler-collected metadata on the API distribution.

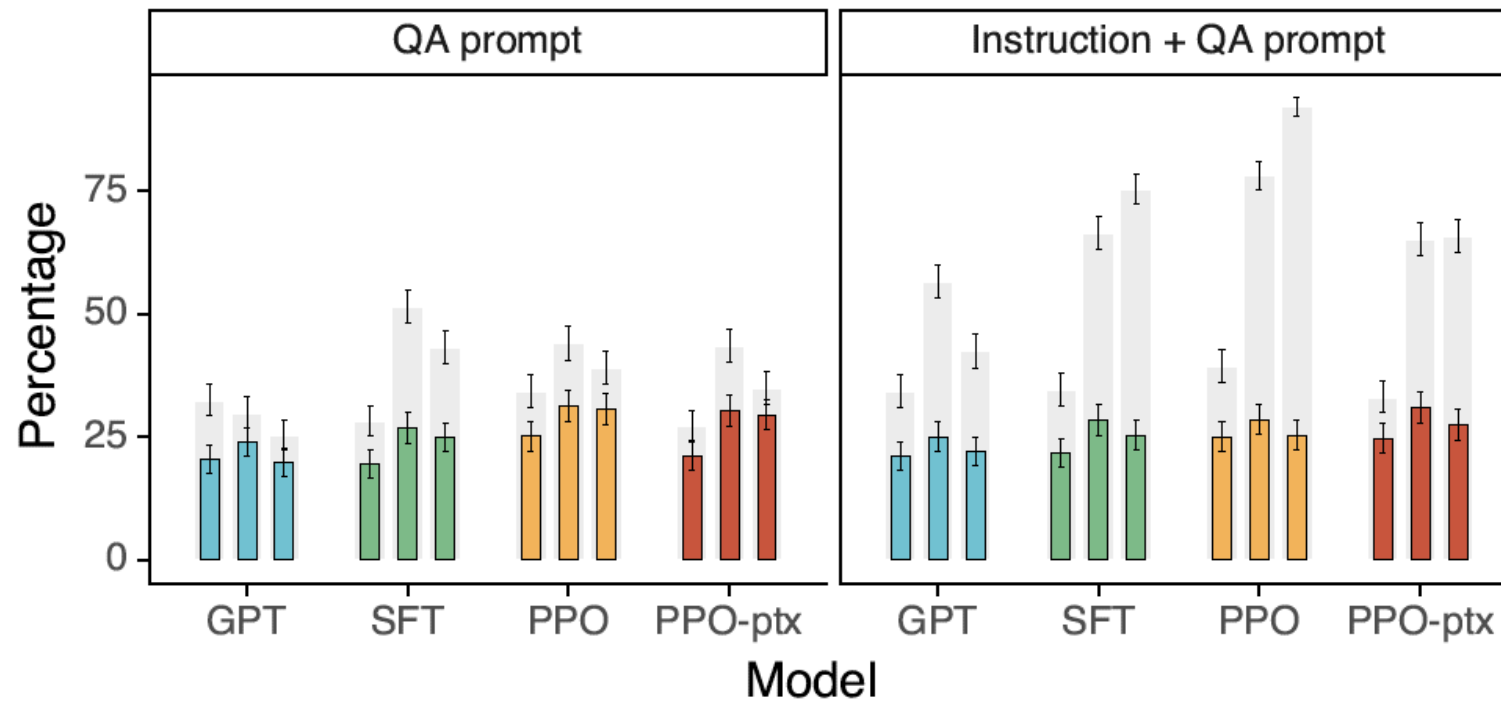
Metadata	Scale
Overall quality	Likert scale; 1-7
Fails to follow the correct instruction / task	Binary
Inappropriate for customer assistant	Binary
Hallucination	Binary
Satisfies constraint provided in the instruction	Binary
Contains sexual content	Binary
Contains violent content	Binary
Encourages or fails to discourage violence/abuse/terrorism/self-harm	Binary
Denigrates a protected class	Binary
Gives harmful advice	Binary
Expresses opinion	Binary
Expresses moral judgment	Binary

- **API distribution**
 - Labelers significantly prefer InstructGPT outputs over outputs from GPT-3
 - Generalizing to the preferences of "held-out" labelers
 - Public NLP datasets are not reflective of how their language models are used
- **Public NLP datasets**
 - Showing improvements in truthfulness over GPT-3
 - Showing small improvements in toxicity over GPT-3, but not bias
 - Minimizing performance regressions on public NLP datasets by modifying their RLHF fine-tuning procedure

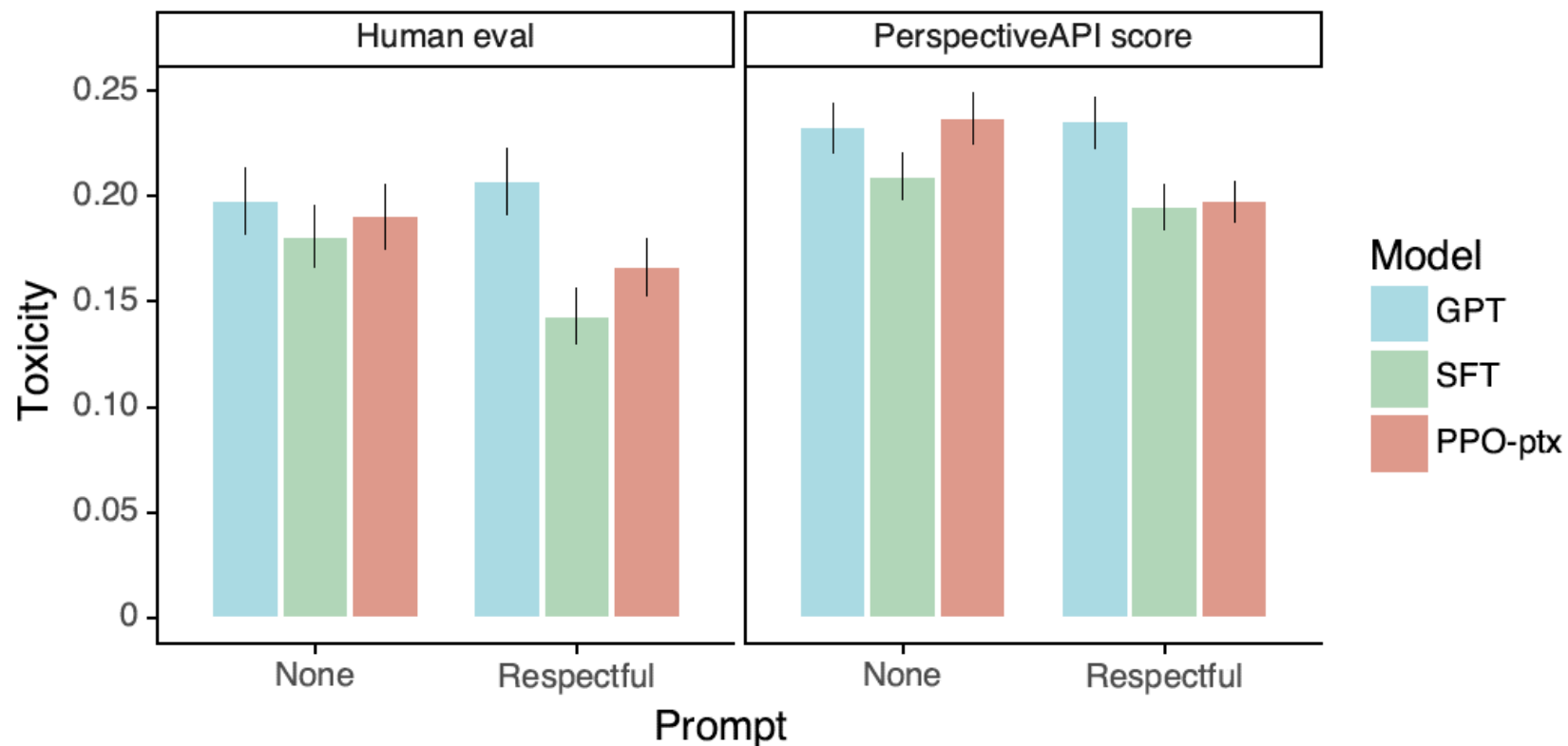
Metadata results on the API Distribution



Results on Truthful Dataset

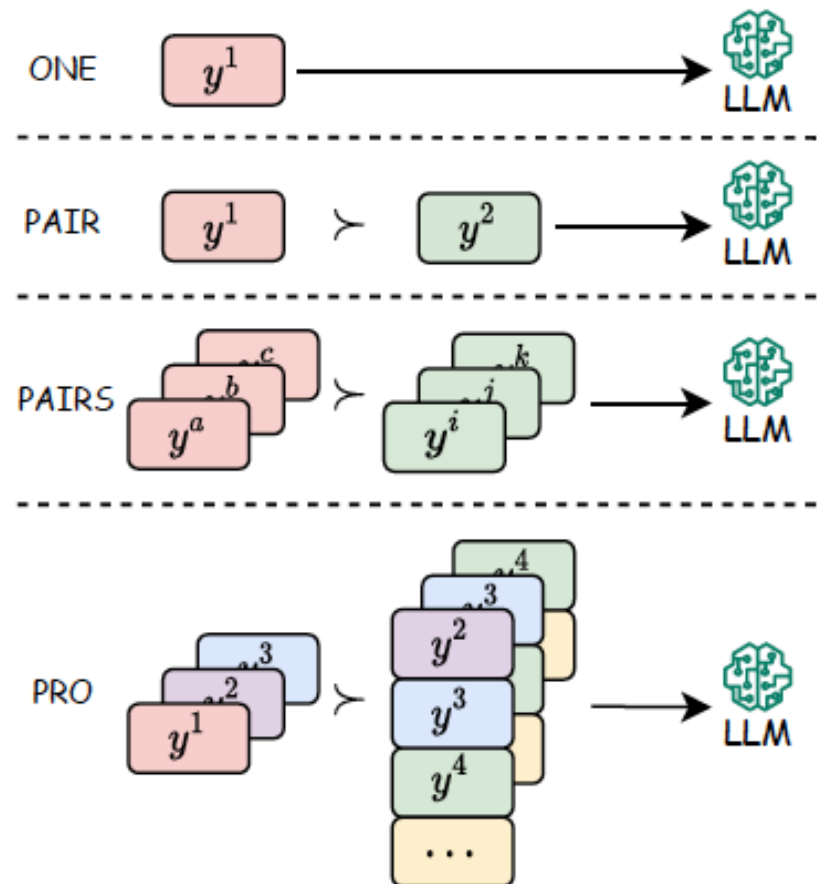


Results on RealToxicityPrompts Dataset



2. PRO: Preference Ranking Optimization

- Reinforcement learning from human feedback (RLHF) encompasses two main drawbacks:
 - RLHF exhibits complexity, instability, and sensitivity to hyperparameters in contrast to SFT
 - Despite massive trial-and-error, multiple sampling is reduced to pairwise contrast, thus lacking contrasts from a macro perspective



Different Supervised Finetuning Paradigms

Human Alignment: Preference Ranking Optimization (PRO)

- From RLHF to PRO
 - Convert listwise ranking to pairwise ranking

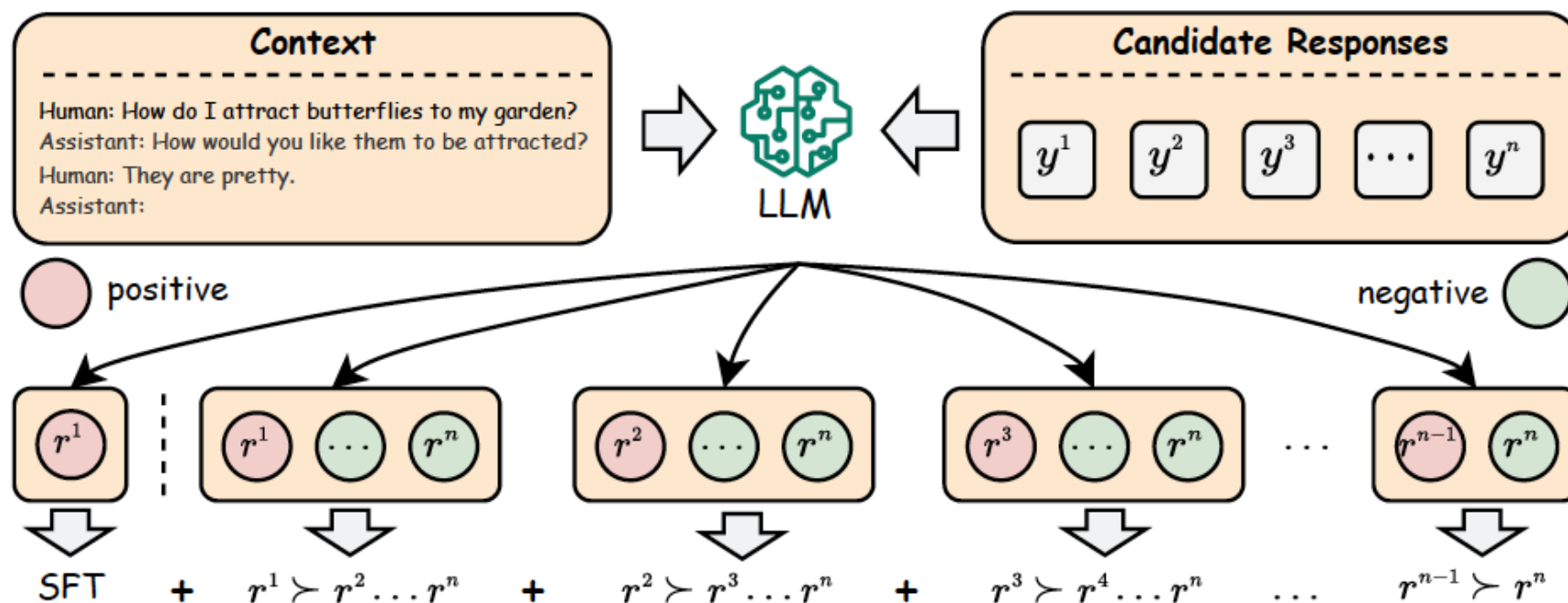
$$y_1 \succ y_2 \succ \cdots \succ y_n \quad \longrightarrow \quad y_1 \succ \{y_2, \cdots, y_n\}$$

$$\mathcal{L}(y_1 \succ \{y_2, \cdots, y_n\}) = -\log \frac{\exp(r_\pi(x, y^1))}{\sum_{i=1}^n \exp(r_\pi(x, y^i))}$$

InfoNCE loss

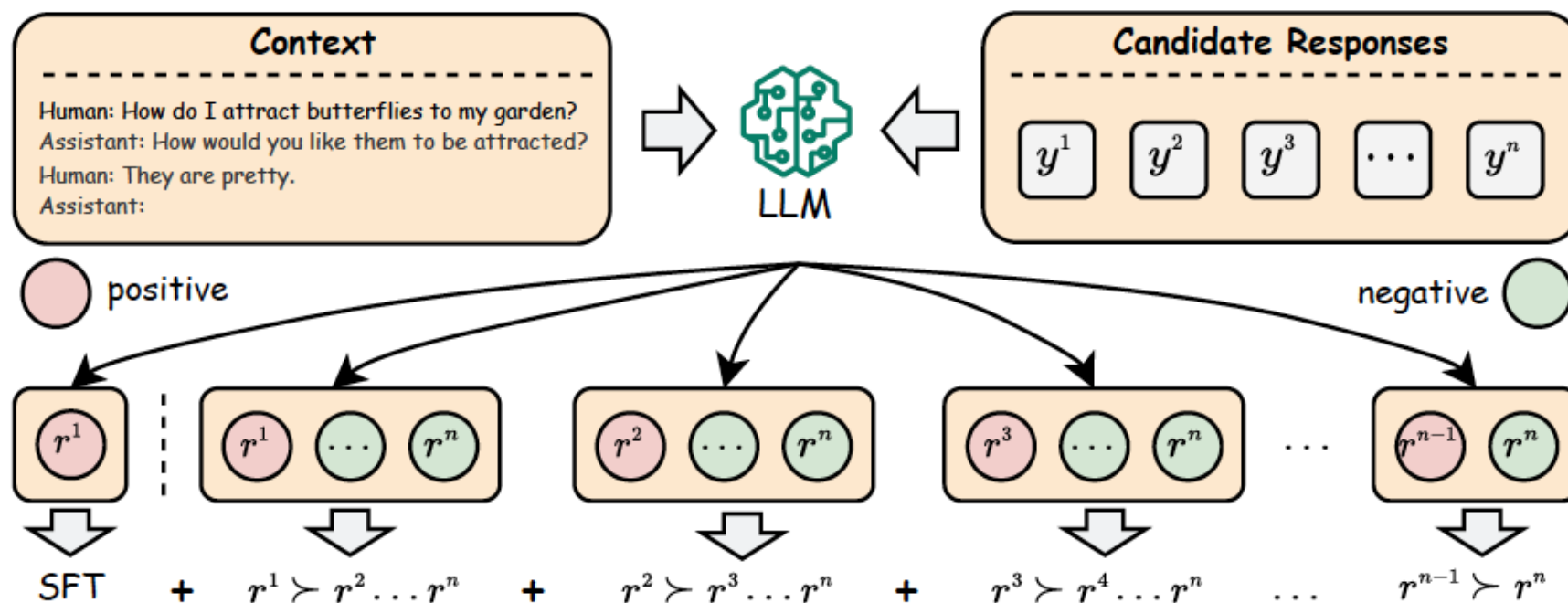
- **Issue:** Does not fully leverage the ranking

Human Alignment: Preference Ranking Optimization (PRO)



The pipeline of PRO for Human Feedback Alignment learning

Human Alignment: Preference Ranking Optimization (PRO)



The pipeline of PRO for Human Feedback Alignment learning

$$\mathcal{L}(y_1 \succ y_2 \succ \dots \succ y_n) = -\log \prod_{k=1}^{n-1} \frac{\exp(r(x, y_k))}{\sum_{i=k}^n \exp(r(x, y_i))}$$

Human Alignment: Preference Ranking Optimization (PRO)

$$\mathcal{L}(y_1 \succ y_2 \succ \dots \succ y_n) = -\log \prod_{k=1}^{n-1} \frac{\exp(r(x, y_k))}{\sum_{i=k}^n \exp(r(x, y_i))}$$

- where

$$r_{\pi_{\text{PRO}}}(x, y^k) = \frac{1}{|y^k|} \sum_{t=1}^{|y^k|} \log P(y_t^k | x, y_{<t}^k)$$

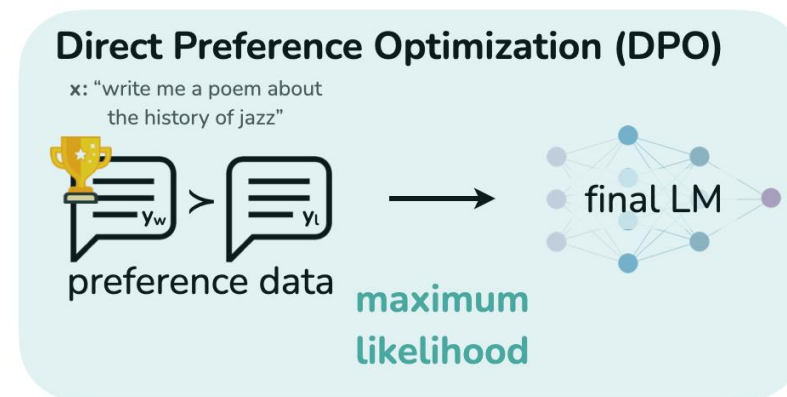
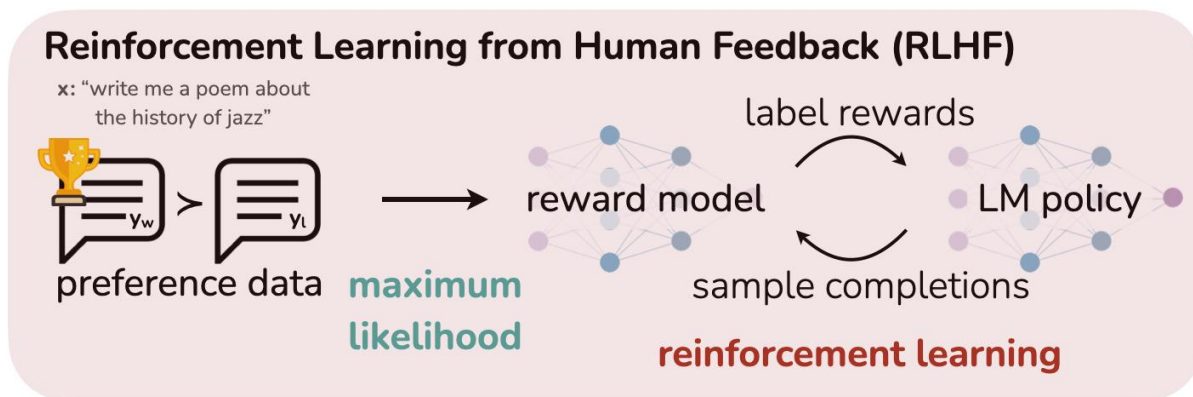
- Instead of optimizing the agent to approximate the RM, PRO enables the agent LLM to be directly trained by the following objective:

$$\mathcal{L}_{\text{PRO}}(y^{1,\dots,n} | x) = \mathcal{L} + \beta \mathcal{L}_{\text{SFT}}$$

- where $\mathcal{L}_{\text{SFT}}$ is the NLL loss of the top 1 candidate and β is the hyperparameter to maintain the balance between text quality and human preference

3. DPO: Direct Preference Optimization

- RLHF is a complex and often unstable procedure,
 - first fitting a reward model that reflects the human preferences,
 - then fine-tuning the large unsupervised LM using reinforcement learning to maximize this estimated reward without drifting too far from the original model
- DPO solves the standard RLHF problem with only a simple classification loss



Direct Preference Optimization

- RLHF Objective:

get high reward, stay close to
reference model

Any reward functions

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi} [r(x, y)] - \beta \mathbb{D}_{KL} [\pi(y | x) \| \pi_{ref}(y | x)]$$

- There is a **closed-form solution** of the above optimization

Direct Preference Optimization

- Deriving Closed-Form Optimal Policy:

$$\begin{aligned}
 & \max_{\pi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi} [r(x, y)] - \beta \mathbb{D}_{KL} [\pi(y | x) \| \pi_{ref}(y | x)] \\
 &= \max_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[r(x, y) - \beta \log \frac{\pi(y | x)}{\pi_{ref}(y | x)} \right] \\
 &= -\beta \min_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y | x)}{\pi_{ref}(y | x)} - \frac{1}{\beta} r(x, y) \right] \\
 &= -\beta \min_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y | x)}{\frac{1}{Z(x)} \pi_{ref}(y | x) \exp \left(\frac{1}{\beta} r(x, y) \right)} - \log Z(x) \right] \\
 &= -\beta \min_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[\mathbb{D}_{KL} \left[\pi(y | x) \left\| \frac{1}{Z(x)} \pi_{ref}(y | x) \exp \left(\frac{1}{\beta} r(x, y) \right) \right\| \right] - \log Z(x) \right]
 \end{aligned}$$

$$\mathbb{D}_{KL}(p \| q) = \mathbb{E}_{u \sim p} \left[\log \frac{p(u)}{q(u)} \right]$$

$$Z(x) = \sum_y \pi_{ref}(y | x) \exp \left(\frac{1}{\beta} r(x, y) \right)$$

Direct Preference Optimization

- RLHF Objective:

get high reward, stay close to
reference model

Any reward functions

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi} [r(x, y)] - \beta \mathbb{D}_{KL} [\pi(y | x) \| \pi_{ref}(y | x)]$$

- Closed-form Optimal Policy:

write optimal policy as function of
reward function

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{ref}(y | x) \exp \left(\frac{1}{\beta} r(x, y) \right)$$

$$\text{with } Z(x) = \sum_y \pi_{ref}(y | x) \exp \left(\frac{1}{\beta} r(x, y) \right)$$

intractable sum over possible response

Direct Preference Optimization

- Closed-form Optimal Policy:

write optimal policy as function of
reward function

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{ref}(y | x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

$$\text{with } Z(x) = \sum_y \pi_{ref}(y | x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

← intractable sum over possible response

- **Rearrange:**

ratio is positive if policy likes response more than
reference model; negative if otherwise.

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{ref}(y | x)} + \beta \log Z(x)$$

Some parameterization of a reward function

Direct Preference Optimization: Putting It Together

A loss function on reward functions

derived from the Bradley-Terry model of human preferences

$$\mathcal{L}_R(r, \mathcal{D}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r(x, y_w) - r(x, y_l))]$$



A transformation
between reward
functions and policy



A loss function on $\mathcal{L}_{DPO}(\pi_\theta; \pi_{ref})$
policy

$$r(x, y) = \beta \log \frac{\pi_\theta(y | x)}{\pi_{ref}(y | x)} + \beta \log Z(x)$$

When substituting, the log Z term cancels, because the loss only care about difference in rewards

Reward of
preferred
response

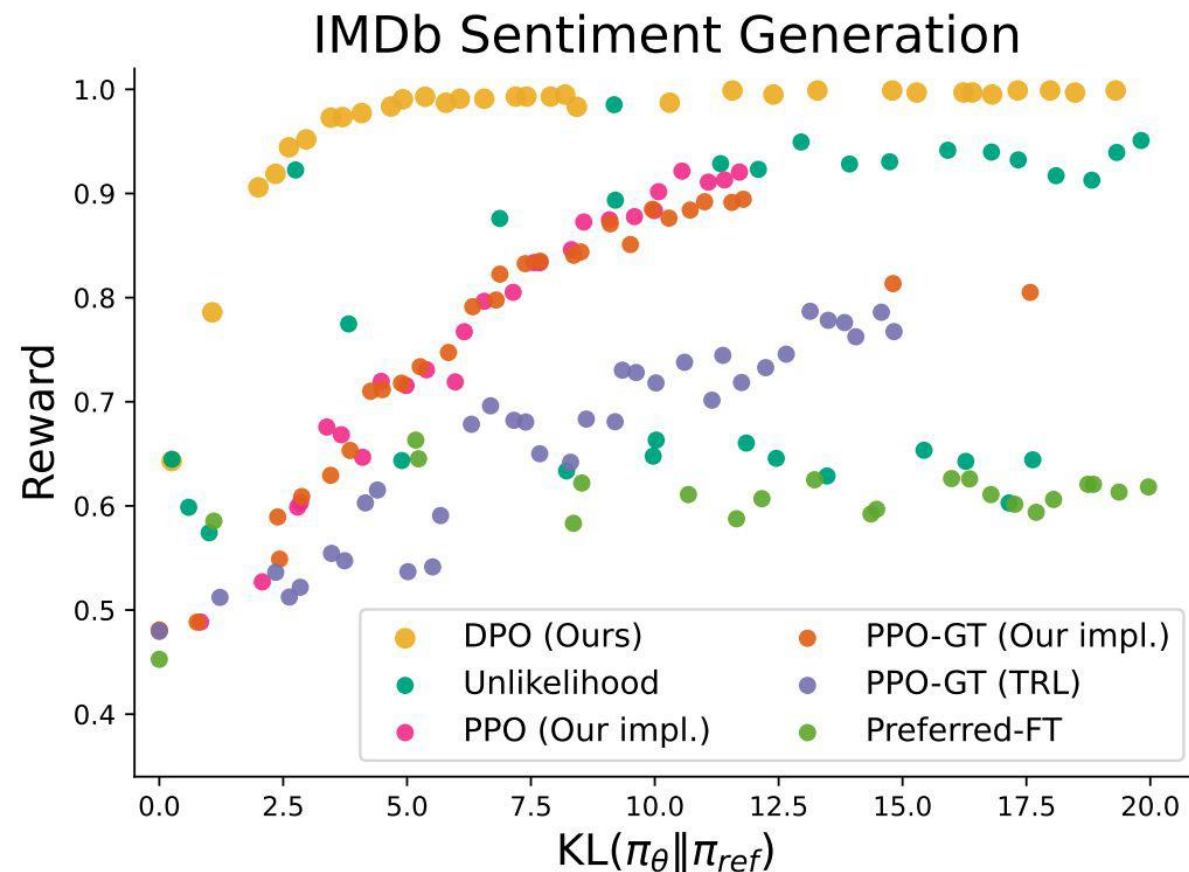
Reward of
dis-
preferred
response

$$= -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{ref}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{ref}(y_l | x)} \right) \right]$$

- **Three tasks**
 - Controlled sentiment generation (IMDb dataset)
 - Summarization (Reddit dataset)
 - Single-turn dialogue (Anthropic Helpful and Harmless dialogue dataset)
- **Evaluation**
 - Controlled sentiment generation: pre-trained sentiment classifier (rewards)
 - Win rates against a baseline policy
 - Use GPT-4 as a proxy for human evaluation of summary quality and response helpfulness
 - Summarization: reference summaries in the test set as baseline
 - Dialog: preferred response in test dataset as baseline

How Efficiently does DPO Trade off Reward & KL?

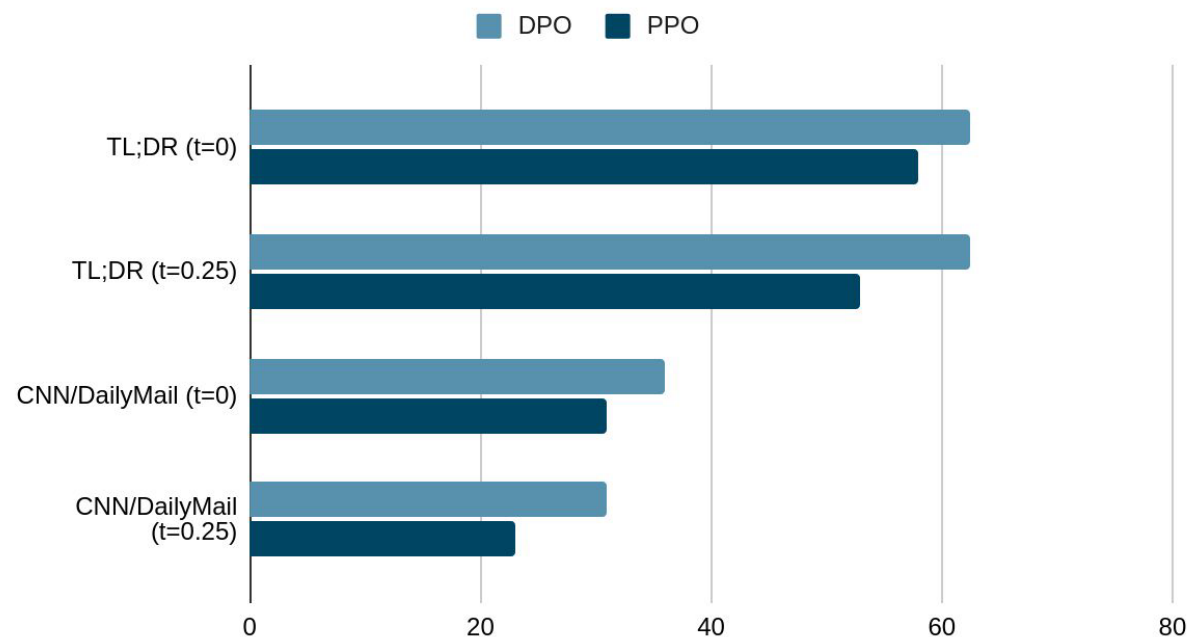
1. Generate positive IMDB reviews from GPT2-XL
2. Use pre-trained sentiment classifier as
3. Create preferences based on Gold RM
4. Optimize with PPO and DPO



DPO vs PPO: Empirics

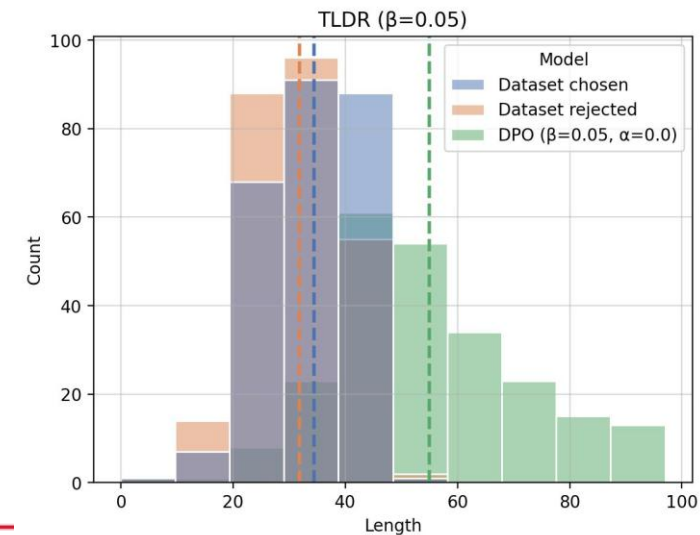
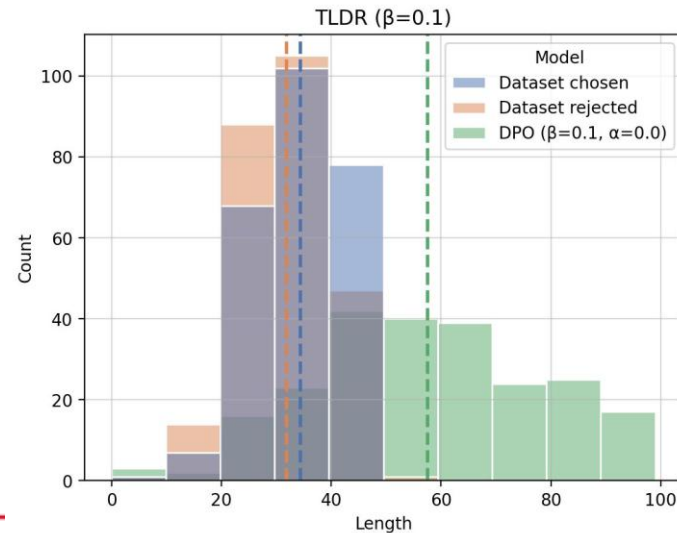
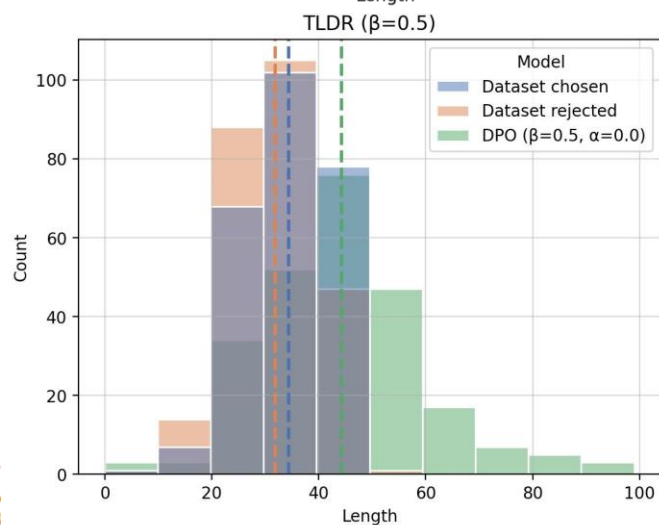
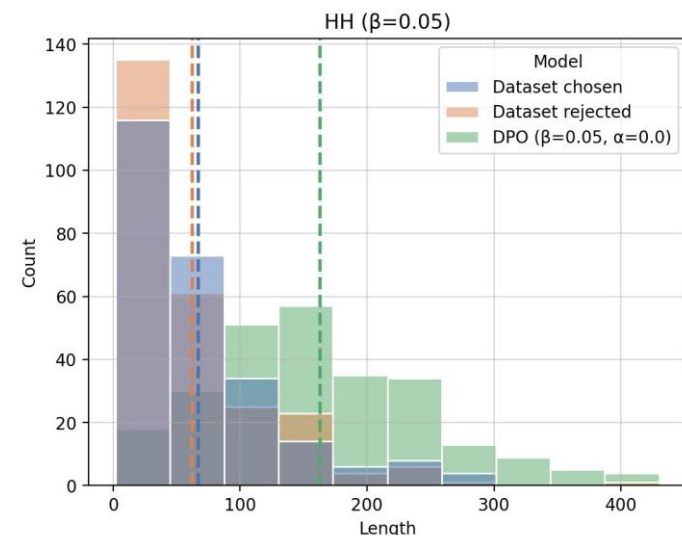
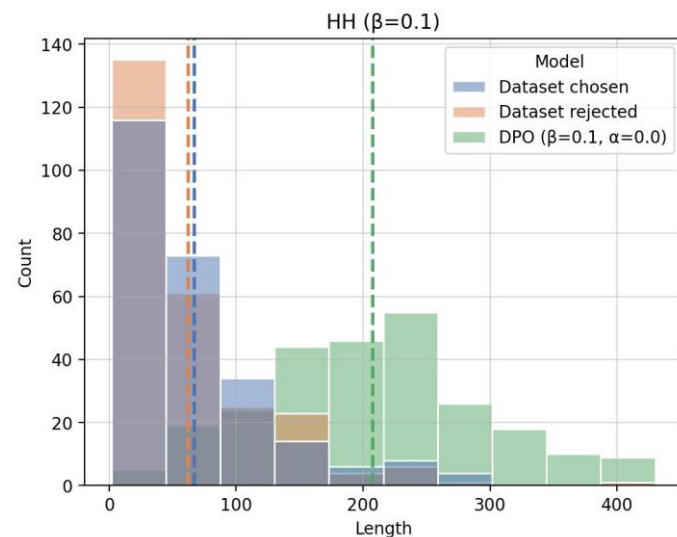
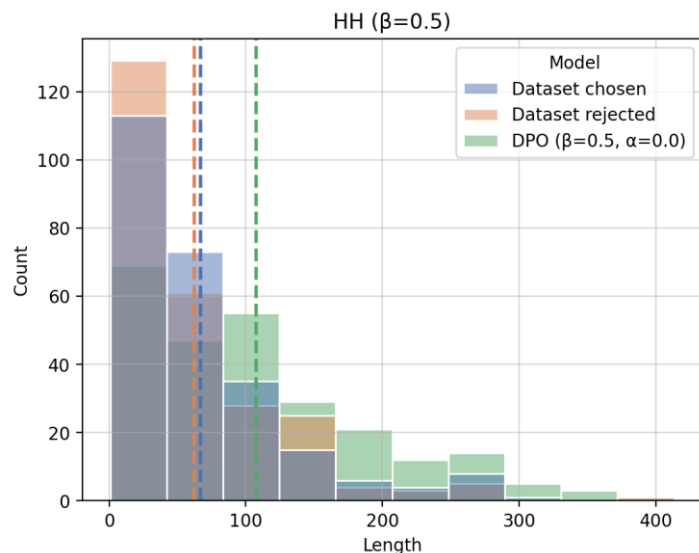
1. DPO is trained only on the Reddit TL;DR feedback data.
2. PPO uses a trained reward function and
3. We evaluate the trained policies on OC task.

Win Rates



- DPO optimizes the same classical RLHF objective
- Is simple and computationally cheap
- Like classical RLHF it is prone to hacking

DPO: Reward Hacking Issue



Length Regularization in DPO

- RLHF objective:

get high reward, stay close to
reference model

Any reward functions

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}} [r(x, y)] - \beta \mathbb{D}_{KL} [\pi(y | x) \| \pi_{ref}(y | x)]$$

- RLHF objective with **length regularization**:

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}} [r(x, y)] - \alpha |y| - \beta \mathbb{D}_{KL} [\pi(y | x) \| \pi_{ref}(y | x)]$$

Length regularization

- **Closed-form optimal policy**

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{ref}(y | x) \exp \left(\frac{1}{\beta} (r(x, y) - \alpha |y|) \right)$$

$$\text{with } Z(x) = \sum_y \pi_{ref}(y | x) \exp \left(\frac{1}{\beta} (r(x, y) - \alpha |y|) \right)$$

Length Regularization in DPO

- Closed-form Optimal Policy:

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{ref}(y | x) \exp \left(\frac{1}{\beta} (r(x, y) - \alpha |y|) \right)$$

with $Z(x) = \sum_y \pi_{ref}(y | x) \exp \left(\frac{1}{\beta} (r(x, y) - \alpha |y|) \right)$

- Rearrange:

ratio is positive if policy likes response more than reference model; negative if otherwise.

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{ref}(y | x)} + \beta \log Z(x) - \alpha |y|$$

Some parameterization of a reward function

DPO with Regularization:

A loss function on reward functions

derived from the Bradley-Terry model of human preferences

$$\mathcal{L}_R(r, \mathcal{D}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r(x, y_w) - r(x, y_l))]$$



A transformation
between reward
functions and policy



A loss function on policy

$$\mathcal{L}_{DPO-R}(\pi_\theta; \pi_{ref})$$

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{ref}(y | x)} + \beta \log Z(x) - \alpha |y|$$

When substituting, the log Z term cancels, because the loss only care about difference in rewards

Reward of
preferred
response

Reward of
dis-
preferred
response

$$= -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{ref}(y_w | x)} - \alpha |y_w| \right) - \left(\beta \log \frac{\pi_\theta(y_l | x)}{\pi_{ref}(y_l | x)} - \alpha |y_l| \right) \right) \right]$$

4. ORPO: Monolithic Preference Optimization without Reference Model

- While recent preference alignment algorithms have demonstrated promising results, SFT remains imperative for achieving successful convergence.
- A minor penalty for the disfavored generation style is sufficient for preference-aligned SF.
- ORPO is a reference model-free monolithic odds ratio preference optimization algorithm that eliminates the necessity for an additional preference alignment phase.

4. ORPO: Monolithic Preference Optimization without Reference Model

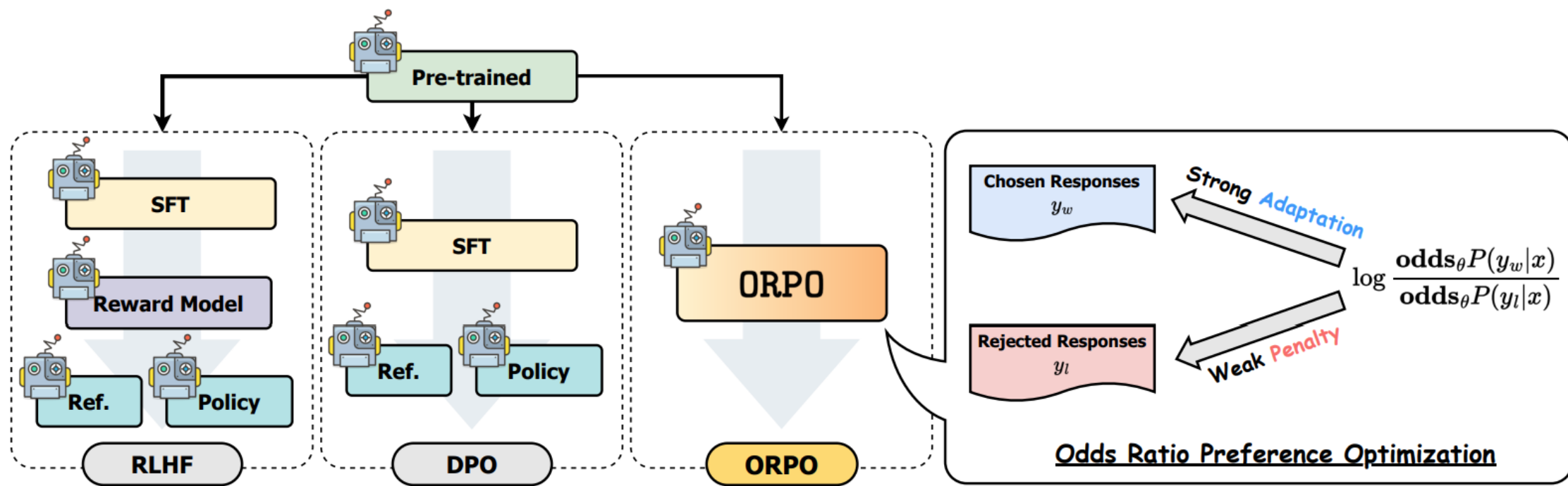


Figure 2: Comparison of model alignment techniques. ORPO aligns the language model *without a reference model* in a single-step manner by assigning a weak penalty to the rejected responses and a strong adaptation signal to the chosen responses with a simple log odds ratio term appended to the negative log-likelihood loss.

4. ORPO: Monolithic Preference Optimization without Reference Model

- The goal of cross-entropy loss is to penalize the model if the predicted logits for the reference answers are low, as shown in Equation 2.

$$\mathcal{L} = -\frac{1}{m} \sum_{k=1}^m \log P(\mathbf{x}^{(k)}, \mathbf{y}^{(k)}) \quad (1)$$

$$= -\frac{1}{m} \sum_{k=1}^m \sum_{i=1}^{|V|} y_i^{(k)} \cdot \log(p_i^{(k)}) \quad (2)$$

- While cross-entropy is generally effective for domain adaptation, there are no mechanisms to penalize rejected responses when compensating for the chosen responses.
- => Therefore, the log probabilities of the tokens in the rejected responses increase along with the chosen responses

4. ORPO: Monolithic Preference Optimization without Reference Model

- Intuitively, $odds_{\theta}(y|x) = k$ implies that it is k times more likely for the model θ to generate the output sequence y than not generating it.

$$odds_{\theta}(y|x) = \frac{P_{\theta}(y|x)}{1 - P_{\theta}(y|x)}$$

- The odds ratio of the chosen response y_w over the rejected response y_l , $OR_{\theta}(y_w, y_l)$, indicates how much more likely it is for the model θ to generate y_w than y_l given input x

$$OR_{\theta}(y_w, y_l) = \frac{odds_{\theta}(y_w|x)}{odds_{\theta}(y_l|x)}$$

4. ORPO: Monolithic Preference Optimization without Reference Model

The objective function of ORPO in Equation 6 consists of two components: 1) supervised fine-tuning (SFT) loss ($\mathcal{L}_{SFT}$); 2) relative ratio loss ($\mathcal{L}_{OR}$).

$$\mathcal{L}_{ORPO} = \mathbb{E}_{(x, y_w, y_l)} [\mathcal{L}_{SFT} + \lambda \cdot \mathcal{L}_{OR}] \quad (6)$$

$\mathcal{L}_{SFT}$ follows the conventional causal language modeling negative log-likelihood (NLL) loss function to maximize the likelihood of generating the reference tokens as previously discussed in Section 3. $\mathcal{L}_{OR}$ in Equation 7 maximizes the odds ratio between the likelihood of generating the disfavored response y_w and the disfavored response y_l . We wrap the log odds ratio with the log sigmoid function so that $\mathcal{L}_{OR}$ could be minimized by increasing the log odds ratio between y_w and y_l .

$$\mathcal{L}_{OR} = -\log \sigma \left(\log \frac{\mathbf{odds}_{\theta}(y_w|x)}{\mathbf{odds}_{\theta}(y_l|x)} \right) \quad (7)$$

4. ORPO: Monolithic Preference Optimization without Reference Model

Model Name	Size	AlpacaEval _{1.0}	AlpacaEval _{2.0}
Phi-2 + SFT	2.7B	48.37% (1.77)	0.11% (0.06)
Phi-2 + SFT + DPO	2.7B	50.63% (1.77)	0.78% (0.22)
Phi-2 + ORPO (<i>Ours</i>)	2.7B	71.80% (1.59)	6.35% (0.74)
Llama-2 Chat *	7B	71.34% (1.59)	4.96% (0.67)
Llama-2 Chat *	13B	81.09% (1.38)	7.70% (0.83)
Llama-2 + ORPO (<i>Ours</i>)	7B	81.26% (1.37)	9.44% (0.85)
Zephyr (α) *	7B	85.76% (1.23)	8.35% (0.87)
Zephyr (β) *	7B	90.60% (1.03)	10.99% (0.96)
Mistral-ORPO- α (<i>Ours</i>)	7B	87.92% (1.14)	11.33% (0.97)
Mistral-ORPO- β (<i>Ours</i>)	7B	91.41% (1.15)	12.20% (0.98)

Table 1: Table of instruction-following abilities of each checkpoint measured through AlpacaEval. While clearly showing the improvements in instruction-following abilities after training with ORPO, it is notable that ORPO models exceed RLHF or DPO models of Llama-2 and Mistral (* indicates the results from the official leaderboard.)

4. ORPO: Monolithic Preference Optimization without Reference Model

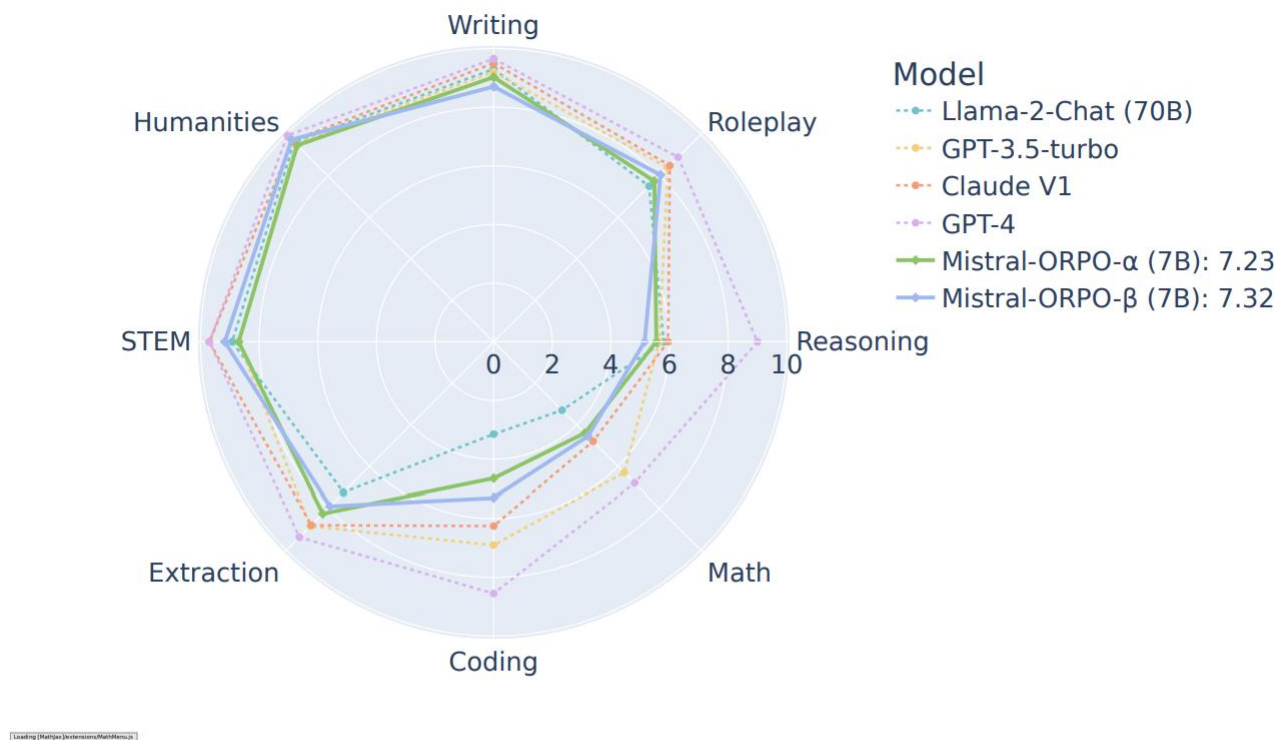


Figure 4: MT-Bench result of Mistral-ORPO- α (7B) and Mistral-ORPO- β (7B) by the category. Further comparison can be found in the Appendix G.

- LLMs with human feedback
 - Goal: align LLM responses with human preferences
 - RLHF: two stages of reward modeling and policy learning
 - DPO: end-to-end policy learning
 - Variants:
 - Listwise rankings versus pairwise rankings
 - Length regularization
- Future works
 - Generalizability of LLM alignment
 - LLM alignment for non-English languages
 - Human-in-the-loop factors